

DSC 40B Discussion 7 Worksheet Solutions

Friday, May 15, 2026

Imagine you are a tutor, and you need to explain something to someone who has only a vague understanding of what's going on. Do your best to get the point across. **ALTERNATE**.

If you're not sure, ask your peer if they know how to explain it, then REPEAT the explanation back. Remember, the point of the exercise is to **vocalize** your thoughts.

If you are both unsure, make sure to come to/tutor for help! Do not use ChatGPT or any other LLM! The point of discussion is to prepare you for the exams.

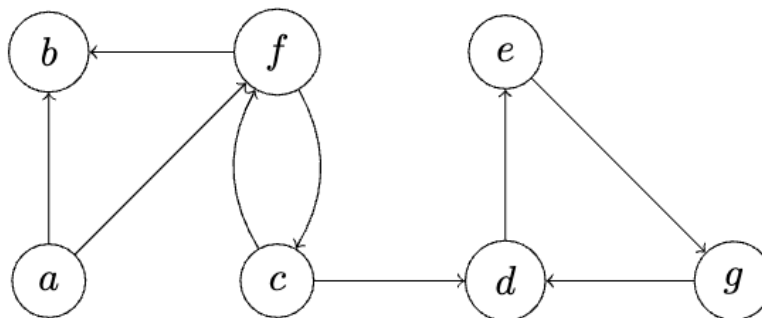
Have one person in the pair open the Discussion 1 Gradescope assignment. After both you **and** your partner finish a problem **and explain it**, answer the corresponding multiple choice/short answer questions for both versions.. You will receive instant feedback on the questions, whether you are correct or not. If your answer is incorrect, please go back to the question to understand it and feel free to ask tutors about anything you are confused about. Once you have finished the entire worksheet or discussion is ending, turn in your assignment and add your partner to the submission.

Quick Icebreaker

Introduce yourself to your partner by sharing name, year, major, etc. Are you looking forward to or dreading the end of the quarter? (Do not spend more than 2 minutes!)

Questions

Question 1:



First Person:

Consider a bread-first search (BFS) starting from **node c**, which nodes are visited and in what order? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

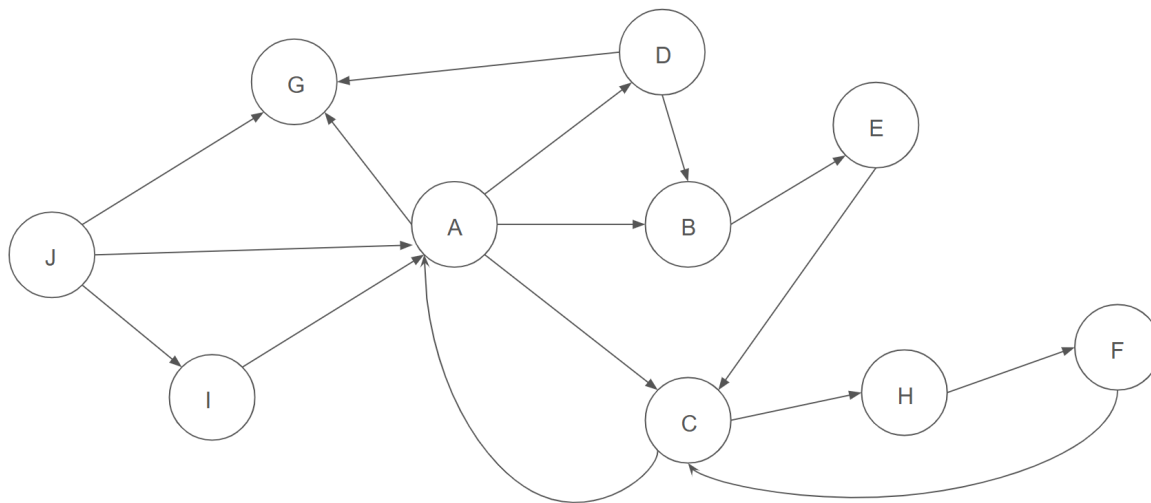
[c,d,f,e,b,g] is the sequence in which nodes are visited. Node a is never visited.

Second Person:

Consider a bread-first search (BFS) starting from **node a**, which nodes are visited and in what order? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

[a,b,f,c,d,e,g] is the sequence in which nodes are visited.

Question 2:



Second Person:

Given the graph above and running BFS from the starting point of **node A**, what is the most amount of nodes in “pending” at the same time?

At the first moment where there were the most amount of nodes in “pending”, which nodes were visited and undiscovered at that time?

Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

The moment where there were most nodes in the pending queue is when the queue was [A,B,C,D,G]. This occurs when BFS is at node A and adding all of A's neighbors to "pending", but before it is marked as "visited".

First Person:

Given the graph above and running BFS from the starting point of **node E**, what is the most amount of nodes in “pending” at the same time?

At the moment where there were the most amount of nodes in “pending”, which nodes were “visited” and “undiscovered” at that time?

Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

The moment where there were most nodes in the pending queue is when the queue was [A,H,B,D,G]. This occurs when BFS is at node A and adding all of A's neighbors to "pending" except C, as its predecessor was C, but before it is marked as "visited". H was already added to pending from visiting C.

Question 3:

First Person:

Given the graph in Question 2 and running BFS from the starting point of **node E**, what is the order that nodes are marked as visited? Use the convention that `graph.neighbors()` returns successors in **alphabetical order**. Explain your reasoning.

The overall sequence of nodes visited is [E,C,A,H,B,D,G,F].

Second Person:

Given the graph in Question 2 and running BFS from the starting point of **node J**, what is the order that nodes are marked as visited? Use the convention that `graph.neighbors()` returns successors in **reverse alphabetical order**. Explain your reasoning.

The overall sequence of nodes visited is [J,I,G,A,D,C,B,H,E,F].

Question 4:

```
1. from collections import deque
2.
3. def full_bfs(graph):
4.     status = {node: 'undiscovered' for node in graph.nodes}
5.     for node in graph.nodes:
6.         if status[node] == 'undiscovered':
7.             bfs(graph, node, status)
8.
9. def bfs(graph, source, status=None):
10.     """Start a BFS at `source`."""
11.     if status is None:
12.         status = {node: 'undiscovered' for node in graph.nodes}
13.
14.     status[source] = 'pending'
```

```

15.     pending = deque([source])
16.
17.     # while there are still pending nodes
18.     while pending:
19.         u = pending.popleft()
20.         for v in graph.neighbors(u):
21.             # explore edge (u,v)
22.             if status[v] == 'undiscovered':
23.                 status[v] = 'pending'
24.                 # append to right
25.                 pending.append(v)
26.         status[u] = 'visited'

```

Look at line 6 in the code above. Whenever that line is executed is it possible ...

Second Person:

... that node is “pending”? Explain your reasoning.

... that node is “visited”? Explain your reasoning.

At line 6, node cannot be "pending" because any previous call to bfs(...) has already fully finished before the loop continues. However, node can be "visited" if it was reached during an earlier BFS call, so full_bfs would skip starting BFS from it again.

First Person:

... that some other node is “pending”? Explain your reasoning.

... that some other node is “visited”? Explain your reasoning.

At line 6, no other node can be "pending" because full_bfs is not currently inside an active BFS traversal, and any previous BFS call ended with an empty pending queue. However, some other node can be "visited" because an earlier BFS call may have already explored and marked reachable nodes as visited.

Question 5:

First Person:

What is the asymptotic runtime of full BFS given a sparsely connected directed graph, where each node has at most a degree of one? Give your answer in terms of $|V|$ and explain your reasoning.

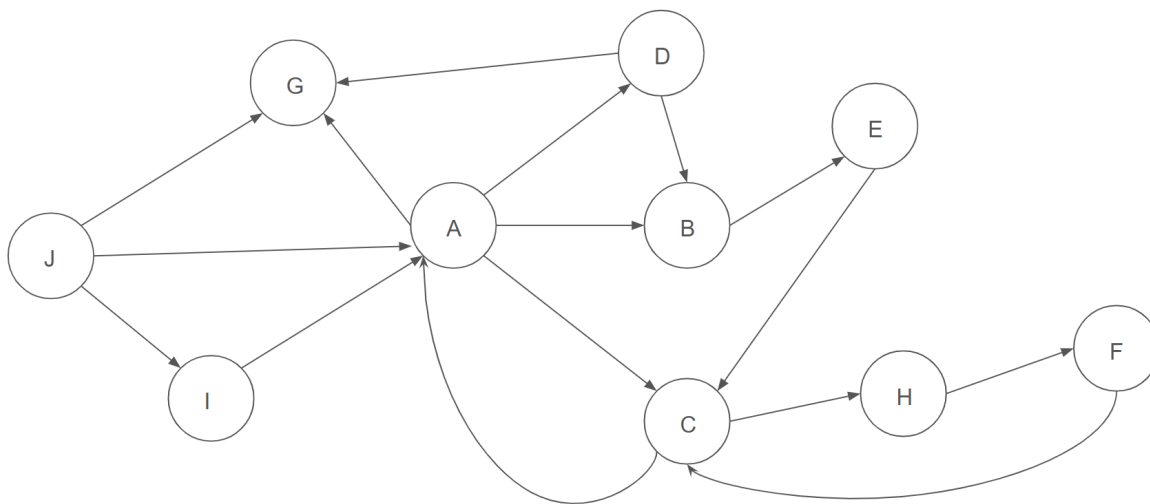
For a sparse graph, $|E| \approx |V|$. Therefore given that the runtime of BFS is $\mathcal{O}(|V| + |E|) \approx \mathcal{O}(|V| + |V|) = \mathcal{O}(|V|)$.

Second Person:

What is the asymptotic runtime of full BFS given a fully connected directed graph, where each node is connected to every other node and itself? Give your answer in terms of $|V|$ and explain your reasoning.

For a fully connected graph, $|E| \approx |V|^2$. Therefore given that the runtime of BFS is $\Theta(|V| + |E|) \approx \Theta(|V| + |V|^2) = \Theta(|V|^2)$.

Question 6:



Second Person:

Given the graph above, what are the distances of the shortest path from **node A** to every other node in the graph.

A: 0
B: 1
C: 1
D: 1
E: 2
F: 3
G: 1
H: 2
I: inf
J: inf

First Person:

Given the graph above, what are the distances of the shortest path from **node E** to every other node in the graph.

A: 2
B: 3
C: 1
D: 3
E: 0
F: 3
G: 3
H: 2
I: inf
J: inf

Question 7:

First Person:

Given the graph in Question 6 and running BFS from the starting point of **node J**, the “pending” queue is currently [**B**, **C**, **D**]. How many nodes currently have a distance of infinity. Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

Currently with pending as [B,C,D], [J,G,A,I] are visited and [E,H,F] are unvisited. The unvisited nodes have infinite distances.

Second Person:

Given the graph in Question 6 and running BFS from the starting point of **node C**, the “pending” queue is currently [**H**, **B**, **D**, **G**]. How many nodes currently have a distance of infinity. Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

Currently with pending as [H,B,D,G], [C,A] are visited and [E,I,J,F] are unvisited. The unvisited nodes have infinite distances.

Question 8:

Second Person:

Given the graph in Question 6 and running BFS from the starting point of **node C**, what are the edges that are in the resulting BFS tree? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

The BFS tree edges are: {(C,A),(C,H),(A,B),(A,D),(A,G),(H,F),(B,E)}.

First Person:

Given the graph in Question 6 and running BFS from the starting point of **node J**, what are the edges that are in the resulting BFS tree? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

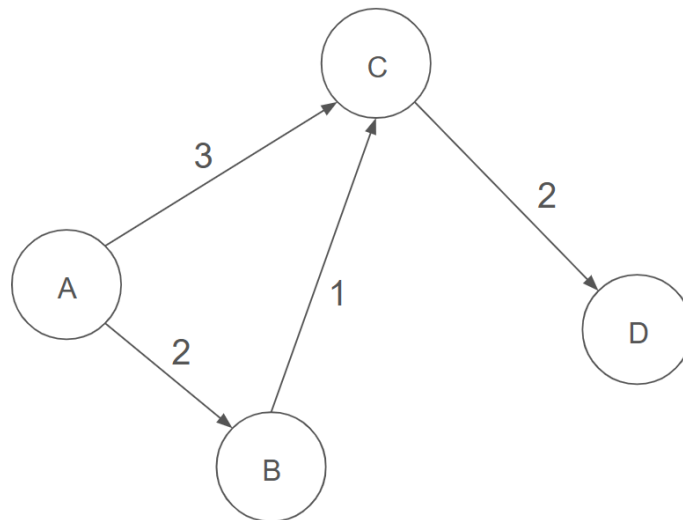
The BFS tree edges are: {(J,A),(J,G),(J,I),(A,B),(A,C),(A,D),(B,E),(C,H),(H,F)}.

Both:

Does the order of `graph.neighbors()` matter for the construction of the BFS tree?

The neighbor order does not change the shortest-path distance found by BFS, but it can change which parent first discovers a node. Since BFS tree edges are based on the first time a node is discovered, different neighbor orders can produce different BFS trees.

Question 9:



First Person:

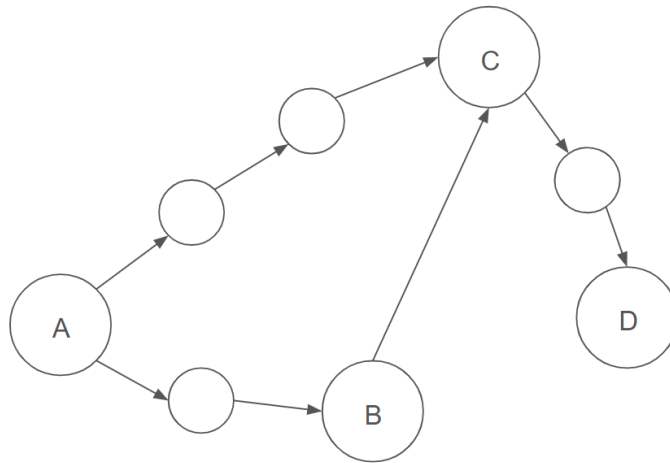
A weighted graph is a graph where the edges have weight values to them, as pictured above.

True or False. BFS will always find the shortest paths from a source to every other node for a weighted graph, given a path between the two nodes exists.

- ☐ True
☒ False

BFS finds the path with the fewest number of edges, not the path with the smallest total weight. In weighted graphs, a path with more edges can have a smaller total weight, so BFS is not guaranteed to find the weighted shortest path. For weighted graphs, you usually need something like Dijkstra's algorithm if all weights are nonnegative.

Second Person:



Now consider the weighted graph has been transformed into an unweighted graph by separating each edge into multiple edges all of weight 1, connected with dummy nodes. An example is shown above.

True or False. BFS will always find the shortest paths from a source to every other for any converted weighted graph like the one above, given a path between the two nodes exists.

- ☒ True
☐ False

After converting the graph, every edge has weight 1, so BFS is now counting the number of edges in the converted graph. Since each original edge of weight w becomes w unit-length edges, the shortest path in the converted graph corresponds to the minimum total weight path in the original graph.